

Общее описание системы

Требуется среда разработки Python, которая работает принципиально иначе, чем все существующие IDE. Вместо хранения кода в текстовых файлах на диске, вся информация о проекте хранится в базе данных. База данных содержит полную структуру проекта, включая файлы, каталоги и все внутренние элементы кода.

Интерфейс должен представлять собой дерево, которое отображает все эти элементы. Каждый элемент в дереве — это не просто строка, а объект, имеющий свойства, версии и срок жизни.

Хранение в базе данных

Весь код разбивается на составные части, каждая из которых сохраняется в отдельной таблице или как отдельная запись. Таблицы соответствуют логическим единицам Python-файла:

1. **Файлы** — общая информация о файле: имя, путь, тип (Python или нет), даты включения и исключения из проекта.
2. **Заголовки файлов** — docstring модуля, который находится в начале файла.
3. **Импорты** — каждая инструкция импорта хранится отдельно с указанием типа (import или from...import), модуля, псевдонима, списка импортируемых имён.
4. **Глобальные переменные модуля** — переменные, объявленные на уровне файла, с их именами, типами, значениями и признаком константы.
5. **Процедуры и функции** — каждая функция — отдельная запись, содержащая имя, признаки асинхронности и генератора, возвращаемый тип и docstring.
6. **Параметры процедур** — для каждой процедуры отдельно хранятся все её параметры с именами, типами, значением по умолчанию, способом передачи и позицией.
7. **Возвращаемые значения** — для каждой процедуры хранятся типы и имена возвращаемых значений.
8. **Локальные переменные процедур** — переменные, объявленные внутри функции, с типами и областью видимости.
9. **Код процедур** — тело функции, хранящееся как массив строк с отступами или как AST-дерево.
10. **Классы** — информация о классах: имя, базовые классы, docstring, признак датакласса.
11. **Свойства классов** — свойства, объявленные через декоратор @property, с их кодом геттеров и сеттеров.
12. **Методы классов** — методы класса, которые имеют те же внутренние элементы, что и процедуры (параметры, возвраты, локальные переменные, код), плюс тип метода (экземплярный, классовой, статический).

Версионность и срок жизни

Каждая запись в любой таблице имеет срок жизни. Это реализовано через два поля: **дата начала активности** и **дата окончания активности**. Если дата окончания не заполнена, запись считается активной в настоящий момент.

Когда элемент создаётся, ему присваивается дата начала. Когда элемент изменяется, старая запись получает дату окончания, а новая создаётся с новой датой начала. Таким образом, полная история изменений сохраняется для каждого элемента отдельно.

Элементы могут быть исключены из проекта — тогда их дата окончания устанавливается на момент исключения. Они перестают отображаться в дереве. Если элемент включается снова, создаётся новая запись с новой датой начала.

Проект можно открыть на любую дату в прошлом. Система покажет только те записи, которые были активны на этот момент. Это позволяет видеть проект таким, каким он был в любой момент времени.

Интерфейс: Первая деревяшка (структура кода)

Интерфейс представляет собой два основных дерева. Первое дерево отображает структуру проекта: каталоги, файлы и их внутреннее содержимое.

Корень проекта

В самом верху дерева находится корневой узел с названием проекта. Он содержит все каталоги и файлы, которые активны на текущий момент.

Каталоги

Каталоги — это виртуальные узлы, которые существуют только в базе данных. Каждый каталог имеет имя, дату создания, дату удаления и дату повторного включения. Каталоги могут быть вложенными. Раскрывая каталог, пользователь видит его содержимое: подкаталоги и файлы.

Файлы общего типа

Все файлы отображаются в дереве как листья или как узлы, если это Python-файлы. Для любого файла отображается его имя и период активности (даты включения и исключения). Файлы, которые не являются Python-скриптами (например, .txt, .json, .yaml), не раскрываются дальше. Они отображаются как конечные элементы с колонками, показывающими время активности и доступ к истории версий. Пользователь может щёлкнуть по такому файлу и увидеть список всех его версий с датами.

Python-файлы

Python-файлы раскрываются. При раскрытии пользователь видит несколько основных

ветвей, соответствующих логическим частям модуля.

Ветвь "Заголовочный комментарий"

Первая ветвь — это заголовочный комментарий, или docstring модуля. Она содержит текст, написанный в начале файла в тройных кавычках. У заголовка есть собственная история: если разработчик меняет описание модуля, создаётся новая версия заголовка, а старая закрывается.

Ветвь "Импорты"

Вторая ветвь — импорты. Каждый импорт отображается как отдельный элемент списка. Для каждого импорта показаны:

- Тип: простой импорт (import module) или импорт из модуля (from module import name).
- Имя модуля.
- Список импортируемых объектов (для from-импортов).
- Псевдоним, если он задан через as.
- Дата добавления или изменения этого импорта.

Импорты могут добавляться, удаляться или изменяться. Каждое изменение сохраняется как новая версия.

Ветвь "Переменные модуля"

Третья ветвь — глобальные переменные, объявленные на уровне файла. Каждая переменная отображается отдельно с указанием:

- Имени.
- Типа (если аннотирован).
- Значения (если задано).
- Признака константы.
- Периода активности.

Ветвь "Процедуры и функции"

Четвёртая ветвь — все функции, объявленные в файле. Каждая функция — отдельный узел, который раскрывается дальше.

При раскрытии конкретной процедуры пользователь видит:

Заголовок процедуры — отображается имя, признаки async def или def, генератор или нет, аннотация возвращаемого типа и docstring.

Входящие параметры — каждый параметр отображается отдельно. Для параметра указаны:

- Имя.
- Тип.

- Способ передачи: по значению или по ссылке.
- Значение по умолчанию, если есть.
- Позиция в сигнатуре.

Возвращаемые значения — если функция возвращает несколько значений, каждое отображается отдельно с типом и позицией.

Локальные переменные — переменные, объявленные внутри функции. Для каждой указаны:

- Имя.
- Тип.
- Область видимости (local, nonlocal, global).

Код процедуры — тело функции, отображаемое как список строк с сохранёнными отступами. Каждое изменение кода — новая версия.

Ветвь "Классы"

Пятая ветвь — все классы в файле. Каждый класс — отдельный узел.

При раскрытии класса пользователь видит:

Описание класса — имя, базовые классы, docstring, признак датакласса.

Свойства — свойства, объявленные через @property. Каждое свойство имеет имя, тип, признак только для чтения и код геттера (и опционально сеттера).

Методы класса — методы, объявленные внутри класса. Методы имеют те же внутренние элементы, что и процедуры: заголовок, входящие параметры, возвращаемые значения, локальные переменные, код. Дополнительно указывается тип метода: экземплярный, классовый или статический.

Все методы и свойства имеют собственную версию и срок жизни.

Интерфейс: Вторая деревьяшка (архитектура разработки)

Вторая деревьяшка отображает процесс разработки: от постановки задачи до конкретных изменений в коде. Это дерево показывает, почему и как были сделаны те или иные изменения.

Задачи

В самом верху дерева находятся задачи. Задача — это исходный запрос от бизнеса или пользователя. Каждая задача содержит:

- Уникальный номер.
- Название.
- Полное описание (промт) — текстом.

- Дату создания.
- Приоритет: критично, высокий, средний, низкий.
- Статус: черновик, утверждена, в работе, выполнена, отменена.
- Дедлайн.
- Связь с родительской задачей, если это уточнение.

К задаче могут быть прикреплены файлы, классы или отдельные процедуры — как контекст, к которому относится задача.

Архитектурные решения

Каждая задача может иметь одно или несколько архитектурных решений. Архитектурное решение — это ответ на вопрос "как мы будем решать эту задачу?".

У архитектурного решения есть:

- Название.
- Полное описание словами — что именно будет сделано и каким образом.
- Выбранный архитектурный подход: монолит, микросервисы, serverless или событийно-ориентированный.
- Технологический стек: список используемых технологий, фреймворков и библиотек.
- Статус: предложено, на рассмотрении, утверждено, отклонено.
- Связи с другими архитектурными решениями, если текущее решение зависит от другого или заменяет его.
- Автор и дата создания.

К архитектурному решению привязываются файлы проекта. Для каждого файла указывается:

- Какой файл затронут (по его идентификатору в базе).
- Тип воздействия: файл будет создан, изменён, удалён или объявлен устаревшим.
- Какие конкретные части файла затрагиваются: список процедур, классов, переменных.
- Обоснование — почему этот файл или его часть нуждается в изменении.

Планы

Архитектурное решение разбивается на планы. План — это последовательность шагов по реализации решения.

У каждого плана есть:

- Название.
- Описание — что делается на этом шаге.
- Порядковый номер.

- Зависимость от другого плана — этот шаг нельзя начать, пока не выполнен предыдущий.
- Исполнитель (назначенный разработчик).
- Статус: ожидает, в работе, выполнено, заблокировано, пропущено.
- Дата начала и дата завершения.

Действия

Каждый план содержит одно или несколько действий. Действие — это атомарная операция над кодом.

Типы действий:

- Создать файл.
- Удалить файл.
- Переименовать файл.
- Создать процедуру.
- Обновить процедуру (изменить код, параметры или типы).
- Удалить процедуру.
- Создать класс.
- Обновить класс.
- Удалить класс.
- Добавить импорт.
- Удалить импорт.
- Обновить глобальную переменную.
- Добавить глобальную переменную.

У каждого действия есть:

- Целевая сущность: идентификатор файла, процедуры, класса, импорта или переменной.
- Описание изменения словами: что именно нужно сделать.
- Новые данные в формате JSON — если это обновление существующей сущности.
- Предусловия — что должно быть выполнено до начала действия.
- Постусловия — что будет после выполнения.
- Ссылка на результат: какая сущность была создана или изменена в итоге.
- Дата выполнения.

Результаты

Каждое действие при выполнении порождает результат. Результат — это фактические изменения, зафиксированные в коде.

В результате сохраняется:

- Ссылка на действие, которое его породило.
 - Какой файл, процедура, класс или импорт был изменён.
 - Тип изменения: создано, обновлено, удалено, объявлено устаревшим.
 - Слепок сущности до изменения (все её данные).
 - Слепок сущности после изменения.
 - Ссылка на версию из таблиц истории.
 - Дата изменения.
-

Связь между двумя деревьями

Два дерева не существуют отдельно. Они представляют два взгляда на один и тот же проект.

Связь работает в обе стороны:

Из второй деревьяшки в первую:

- Выбирая задачу, пользователь видит список файлов и частей кода, которые эта задача затрагивает.
- Раскрывая архитектурное решение, пользователь видит все файлы и их части, включённые в это решение.
- Раскрывая план, пользователь видит, какие файлы будут созданы или изменены на этом шаге.
- Раскрывая действие, пользователь видит конкретную процедуру или класс, который будет изменён.

Из первой деревьяшки во вторую:

- Кликая на файл, пользователь видит все задачи, архитектурные решения и планы, в которых этот файл участвует.
 - Кликая на процедуру, пользователь видит, для какой задачи она была создана или изменена, каким планом и действием.
 - Кликая на любую версию процедуры, пользователь видит, какое действие привело к созданию этой версии.
-

Интеграция с IDE

Система должна работать с существующими средами разработки, в частности с PyCharm. Для этого необходимо реализовать файловый шлюз, который будет эмулировать файловую систему.

При открытии проекта система собирает из базы данных все активные файлы и их содержимое на текущую дату. Собранные файлы материализуются во временную директорию как обычные .py-файлы. PyCharm открывает эту директорию как проект и работает с ней стандартным образом.

Когда разработчик сохраняет файл, система определяет, какие процедуры или классы изменились. Вместо перезаписи всего файла, система обновляет только изменённые записи в базе данных. Старые версии закрываются, новые создаются.

При сохранении система также проверяет, связано ли изменение с какой-либо задачей. Если разработчик указал номер задачи, изменение автоматически привязывается к этой задаче и текущему плану.

Работа с искусственным интеллектом

Вся архитектура построена для эффективной работы с ИИ.

Когда разработчик просит ИИ помочь с задачей, система формирует запрос, содержащий:

- Только ту процедуру или класс, которые нужно изменить.
- Её параметры, типы, локальные переменные.
- Импорты, необходимые для контекста.
- Связанные типы из других файлов (если они указаны в аннотациях).

ИИ получает не весь файл, а только необходимый контекст. Это ускоряет обработку и снижает стоимость запросов.

ИИ-агенты используются на всех этапах:

1. **ИИ-архитектор** принимает задачу, анализирует существующий код и предлагает архитектурное решение с указанием файлов и их частей для изменения.
2. **ИИ-планировщик** разбивает архитектурное решение на последовательные шаги с атомарными действиями.
3. **ИИ-кодер** выполняет каждое действие: генерирует код новой процедуры или обновляет существующую.
4. **ИИ-рефактор** улучшает код по запросу разработчика.
5. **ИИ-тестировщик** генерирует тесты для процедур и классов.

Все изменения, созданные ИИ, сохраняются в базу данных как отдельные версии с указанием автора "AI". Они проходят те же проверки и сохраняют ту же историю, что и изменения, сделанные человеком.

Итог

Система представляет собой две взаимосвязанные иерархии: структуру кода и архитектуру разработки. Всё хранится в базе данных с полной историей на уровне каждого элемента. Интерфейс позволяет переключаться между двумя видами и видеть связи между задачами и кодом. ИИ работает точно, только с необходимыми элементами, что делает его эффективным и быстрым. PyCharm используется как клиент через файловый шлюз, сохраняя привычный рабочий процесс.